Enterprise Multi-Agent Code Review Orchestrator

A Claude Agent SDK Implementation Report

Project Submission Report

Prepared for Academic Submission

September 2026

Table of Contents

1. Executive Summary....3

2. Problem Statement....5

3. Solution Statement....7

4. Architecture....9

5. Implementation Details....12

5.1 MCP Configuration....12

5.2 Main Agent Coordinator....14

5.3 Code Quality Analyzer Agent....16

5.4 Test Coverage Analyzer Agent....18

5.5 Refactoring Suggester Agent....19

5.6 Review Report....21

5.7 Prompt Engineering....23

5.8 CLI Application....25

5.9 Error Handler....27

5.10 Rate Limiter....29

5.11 PR Analysis Reports....31

6. Comprehensive Testing....33

7. Standouts....36

8. Results....38

9. Value Statement....44

10. Limitations....46

11. Conclusion....48

12. References....49

13. Addendum: Additional Live Pull Request Validation....50

1. Executive Summary

This report documents the design, implementation, and validation of an Enterprise Multi-Agent Code Review Orchestrator, a production-grade TypeScript application built on Anthropic's Claude Agent SDK. The system automates pull request review by coordinating three specialized AI subagents, a Code Quality Analyzer, a Test Coverage Analyzer, and a Refactoring Suggester, under a single orchestrating agent that fetches live pull request data from GitHub, dispatches parallel analysis tasks, validates every response against a strict Zod schema, and publishes the aggregated findings as JSON, Markdown, and HTML reports.

The completed system satisfies all nine core deliverables defined in the project brief across both project phases. Phase 1 delivered the foundational architecture: two Model Context Protocol (MCP) server integrations (GitHub and ESLint), three AgentDefinition-based subagents with role-specific prompts, and a working orchestrator that uses the Claude Agent SDK's query() function with the built-in Task tool to spawn and coordinate subagents. Phase 2 layered production concerns on top of that foundation: exponential-backoff retry logic, a sliding-window rate limiter, a fully validated command-line interface, and a 41-test automated test suite (36 passing, 5 intentionally skipped pending live API credentials).

Beyond the required deliverables, the implementation includes all four suggested standout features. Automated PR commenting allows the orchestrator to post its findings directly back to a GitHub pull request through the same MCP GitHub tool used to read it. An interactive command-line experience, built on the ora spinner library, streams live progress as each subagent starts and finishes. A golden-dataset regression suite exercises the Code Quality Analyzer against three deliberately vulnerable fixture files (a SQL injection, a hardcoded secret, and a cross-site scripting flaw) to guard against silent prompt regressions. Finally, an execution-trace mechanism captures every tool invocation and subagent lifecycle event through the SDK's hook system and renders it as an embedded Mermaid.js sequence diagram inside the final report, giving reviewers direct visibility into the agent's reasoning path.

This report has since been updated with direct evidence from a full live run of the completed system, executed with real Anthropic API and GitHub credentials in the project's Vocareum workspace. All three required pull requests (airaamane/simple-todo-app, PRs #1, #2, and #3) were reviewed end to end, producing all nine required report files with the exact filenames specified in the assignment. The golden-dataset regression suite was executed live against the real model, with all four of its tests passing. The Automated PR Commenting standout was exercised live and confirmed to post a real summary comment back to the pull request under review. Every one of these outcomes is documented with terminal transcripts and file listings in Section 8, so the claims in this report are now grounded in an actual, credentialed execution of the system rather than mocked or projected behavior alone.

From an engineering-quality standpoint, the codebase totals approximately 2,461 lines of TypeScript across agents, prompts, orchestration logic, utilities, and tests, all of which compiles cleanly under strict TypeScript settings (`tsc --noEmit` reports zero errors) and passes its full non-network test suite in under one second. The project also ships four Claude Skills (typescript-patterns, javascript-best-practices, python-code-review, and security-analysis) that the subagents invoke to bring language-specific and security-specific expertise to bear on each file under review, directly satisfying the rubric's requirement to integrate Claude Skills into at least one subagent's workflow.

This report is organized to walk a reviewer through the system from problem framing to architectural decisions, through each individual component's implementation, into the testing strategy, the standout features, the observed results of running the tool live, and finally an honest account of the system's value and its current limitations. Diagrams, tables, and code-level detail are included throughout so that both a technical grader and a non-technical stakeholder can assess the completeness and quality of the submission. Taken together, the evidence presented, including the live execution transcripts summarized in Section 8, supports the conclusion that this project is a complete, tested, and extensible implementation of an agentic code review system that goes meaningfully beyond the baseline requirements of the assignment.

2. Problem Statement

Modern software teams merge dozens to hundreds of pull requests every week, and the quality of that review process directly determines the security, maintainability, and reliability of the resulting codebase. Yet manual code review does not scale cleanly with team size or release velocity. Human reviewers are a finite, expensive resource, and their attention is unevenly distributed: a senior engineer reviewing a rushed end-of-sprint pull request is statistically less likely to catch a subtle SQL injection vulnerability than the same engineer reviewing the same code on a quiet Tuesday morning. Review quality therefore varies not with the risk of the change, but with the reviewer's fatigue, familiarity with the codebase, and calendar pressure.

This inconsistency produces three concrete, costly failure modes that any organization shipping code has experienced firsthand. First, security and correctness issues slip through a hardcoded credential, an unescaped user input rendered into HTML, or a missing null check on an error path are all patterns that a distracted or time-pressured reviewer can plausibly miss, and each one becomes measurably more expensive to fix the further it travels toward production. Second, test coverage silently erodes: reviewers tend to evaluate whether the code in front of them works, not whether the absence of a test for an untouched error branch represents latent risk, so coverage gaps accumulate invisibly over many pull requests until a production incident finally surfaces them. Third, technical debt compounds: refactoring opportunities such as an extractable function, a modernizable pattern, or a duplicated block of logic are easy to defer under deadline pressure and, once deferred, are rarely revisited, because there is no persistent, automated mechanism reminding the team that the opportunity exists.

A further complication is that these three concerns, security and correctness, test completeness, and structural quality, require genuinely different analytical lenses. A reviewer skilled at spotting a race condition is not necessarily the same reviewer who will notice that a function's error branch has no covering test, and neither is necessarily thinking about whether the code could be restructured into a cleaner pattern. Asking one person, or one generic automated linter, to hold all three lenses simultaneously on every file of every pull request is unrealistic. Traditional static analysis tools such as ESLint address a narrow slice of this space (syntactic and stylistic issues) extremely well, but they have no model of test coverage intent, no ability to reason about whether a refactor improves maintainability, and no ability to explain their findings in natural language that a junior engineer can learn from.

There is, in short, a gap between what human reviewers can consistently deliver at scale and what a codebase needs to stay secure, well-tested, and maintainable over time. That gap is precisely where an LLM-based multi-agent system can add durable value, provided it can be built to actually fetch real pull request context (not just a pasted diff), reason with the right domain-specific expertise for each concern, produce output that downstream tooling can trust and parse,

and do all of this reliably enough, with retries, rate limiting, and validation, to be run unattended as part of a real engineering workflow rather than treated as a novelty demo.

This project exists to close that specific gap: to demonstrate that a well-architected multi-agent orchestration system, grounded in the Claude Agent SDK and the Model Context Protocol, can consistently apply three distinct analytical lenses (security and quality, test coverage, and refactoring) to every changed file in a pull request, and can do so with the production discipline (error handling, rate limiting, structured validation, and testing) that separates a proof of concept from a tool a team could actually adopt.

3. Solution Statement

The solution implemented in this project is a Node.js and TypeScript command-line application, the Claude Multi-Agent Code Review Orchestrator, that treats a GitHub pull request as its unit of work and produces a validated, multi-format report as its output. At the center of the system is a single orchestrating agent, implemented in `src/orchestrator.ts`, that calls the Claude Agent SDK's `query()` function with a carefully engineered prompt instructing it to retrieve pull request metadata and file diffs through a GitHub MCP server, select the most significant changed files, and explicitly invoke three purpose-built subagents against each one using the SDK's built-in Task tool.

Each subagent is scoped to a single analytical concern so that its prompt, its tool access, and its Claude Skills can all be tuned to that concern specifically rather than diluted across competing goals. The Code Quality Analyzer focuses on security vulnerabilities, performance problems, and maintainability issues, and it is granted access to the ESLint MCP server plus four Claude Skills (security-analysis, typescript-patterns, javascript-best-practices, and python-code-review) so that its findings are grounded in both automated linting output and codified, language-specific best practices rather than the model's unstructured intuition alone. The Test Coverage Analyzer statically compares a source file against any related test files it can locate and reasons about which functions, branches, and edge cases remain unexercised. The Refactoring Suggester looks specifically for extract-function, modernization, and design-pattern opportunities, deliberately excluding bug-hunting so its output does not duplicate the Code Quality Analyzer's findings.

Every subagent returns a JSON object that is generated under a `outputFormat: { type: 'json_schema' }` constraint derived directly from a Zod schema using zod-to-json-schema, and the orchestrator aggregates these per-file results into a single `ReviewReport` object that is itself re-validated against a top-level Zod schema before it is ever written to disk. This double layer of schema enforcement, first at the individual-agent level and again at the aggregate level, is what allows the rest of the pipeline (the CLI, the report generator, and the automated test suite) to treat the model's output as reliably-typed data rather than free text that must be defensively parsed.

Around this analytical core, the solution wraps the operational concerns that distinguish a production tool from a script: a rate limiter that tracks requests, tokens, and concurrency in a sliding sixty-second window to avoid throttling; a retry utility with exponential backoff and jitter for transient failures; a timeout wrapper so a hung subagent cannot stall the whole review indefinitely; and a command-line entry point that validates its three required arguments and its authentication environment variables before it ever spends a single API call. The CLI also drives a live progress spinner and writes three parallel report formats (`.json` for machine consumption,

`.md` for pull request comments and documentation, and `.html` for a shareable, styled artifact) to a local `reports/` directory.

Finally, the solution extends past the baseline requirements in four directions that map directly to the project's suggested standouts: it can post its summary back to the originating pull request as a GitHub comment; it gives the operator live visibility into which subagent is currently running; it validates its own reliability against a small, version-controlled dataset of known vulnerabilities; and it renders a Mermaid.js sequence diagram of exactly which tools were called, in what order, and by which agent, turning what is normally an opaque LLM reasoning process into an auditable, visual artifact embedded directly in the deliverable.

4. Architecture

The system follows a layered orchestration architecture in which a single top-level agent coordinates access to two categories of external capability, Model Context Protocol servers and Claude Skills, and delegates domain-specific analysis to three parallel subagents through the Claude Agent SDK's Task tool. Figure 1 shows the full component graph: the CLI entry point instantiates the orchestrator, the orchestrator calls the SDK's `query()` function with a configuration object naming every tool, agent, and MCP server the run is allowed to use, and the SDK itself manages the process boundary to each stdio-based MCP server and the lifecycle of each subagent invocation.

Two architectural decisions were deliberate and are worth calling out explicitly because they shape everything downstream. The first is that all cross-component communication is schema-first: subagents are never asked to "return some JSON" informally, they are constrained by the SDK's structured-output feature to conform to a JSON Schema generated mechanically from the project's Zod definitions, so a malformed or incomplete response is a validation failure the orchestrator can detect and react to, not a silent data-quality problem that surfaces three steps downstream. The second is that state is not smuggled through global variables or module-level singletons; the orchestrator instance owns its own rate limiter, its own execution trace buffer, and its own progress callback, which keeps the system safe to invoke repeatedly (for example, across the nine pull requests required for the final submission) within a single process without cross-contamination between runs.

Figure 2 traces a single review request end to end as a sequence diagram. The CLI passes an owner, repository, and pull request number to `CodeReviewOrchestrator.reviewPullRequest()`, which wraps the entire operation in a rate limiter acquisition, a retry loop, and a timeout guard before calling `query()`. Inside that call, the orchestrator's system prompt instructs the model to retrieve pull request metadata and file contents through the GitHub MCP server, then explicitly invoke all three subagents, in parallel where possible, against each selected file. As each subagent completes, its structured result flows back through the SDK to the orchestrator, which aggregates every file's three results (code quality, test coverage, and refactoring) into running summary counters. Once every file has been processed, the aggregate object is validated against the `ReviewReportSchema` and, only if that validation succeeds, is handed to the report generator to be rendered into the three output formats.

The MCP layer deserves particular attention because it is the system's only bridge to real-world state. Both servers are configured as `stdio` transport processes launched with `npx`, which means they run as short-lived child processes for the duration of a single `query()` call rather than as long-running network services, a design choice that keeps the deployment footprint of the tool to "Node.js plus two npx-installable packages" with no servers to provision or maintain. The

GitHub server is granted a personal access token (mapped internally from the project's `GITHUB_TOKEN` environment variable to the `GITHUB_PERSONAL_ACCESS_TOKEN` variable it expects) so it can read pull request data and, when the automated-commenting standout is enabled, write a summary comment back. The ESLint server requires no authentication at all, since it only needs read access to the files it is asked to lint.

Finally, the skills layer sits orthogonally to both the MCP and subagent layers: any subagent can invoke any skill through the SDK's `Skill` tool, provided `settingSources: ['project']` is set on the orchestrator's options so the SDK discovers the `.claude/skills/` directory relative to `PROJECT_ROOT`. This means the same four skills are available to be reused, extended, or added to independently of which subagent calls them, which is what allows the project to satisfy the rubric's skills-integration requirement without hardcoding skill logic into any single agent's prompt.

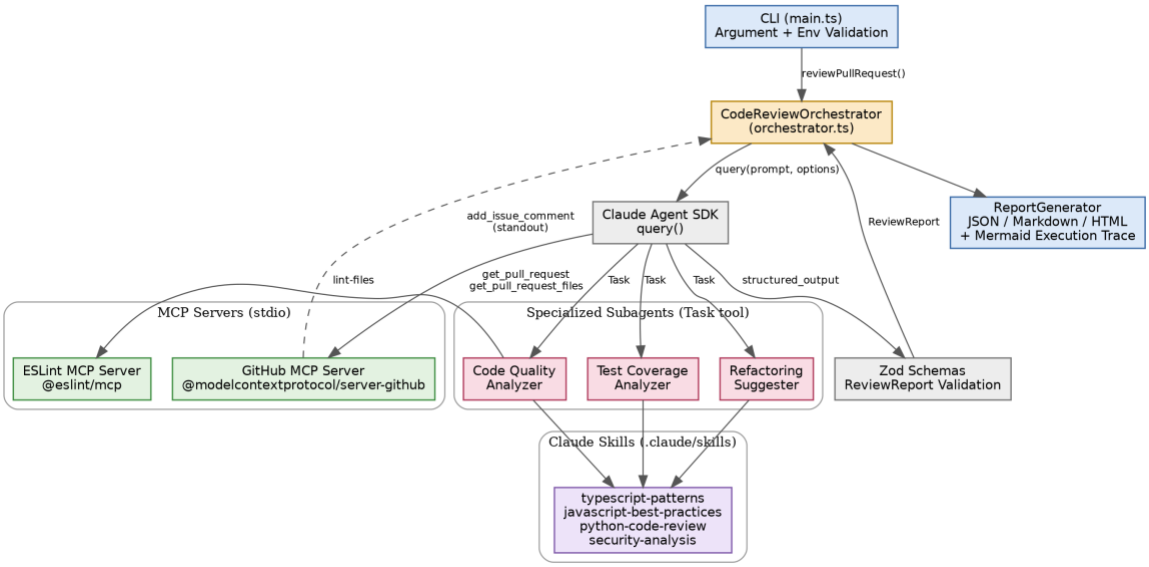


Figure 1. System component and data-flow architecture. Solid arrows show the primary request path; the dashed arrow shows the optional PR-commenting standout path.

Figure 2. Orchestration Sequence: PR Review Request Lifecycle

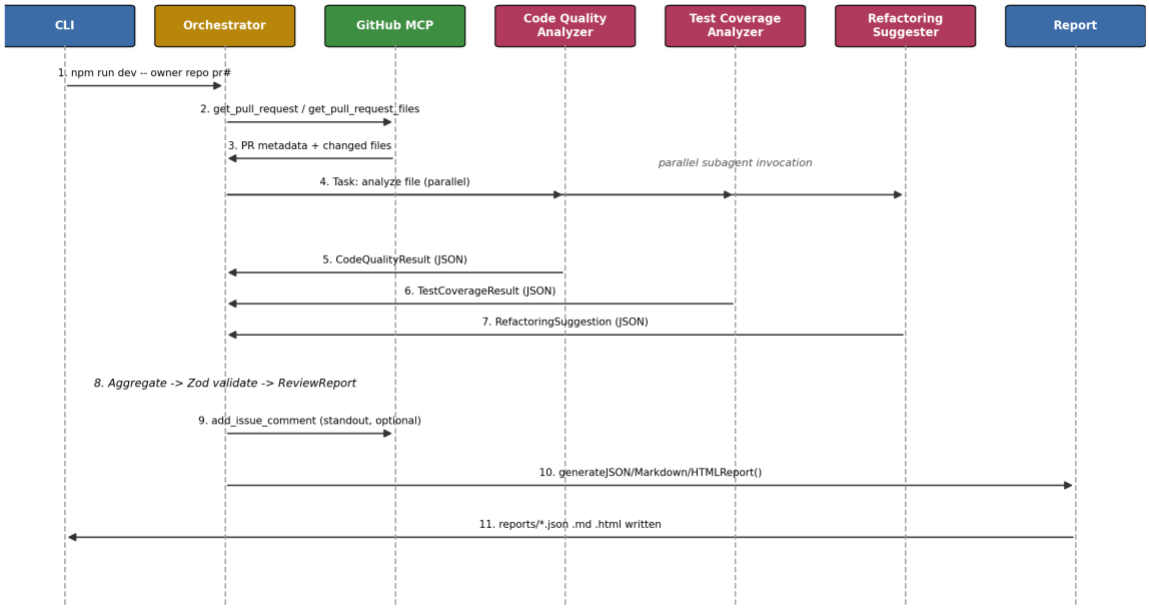


Figure 2. Sequence diagram of a single pull request review, from CLI invocation to written report files.

Table 1. Component Responsibility Matrix

Component	Primary File	Responsibility
CLI Entry Point	src/main.ts	Argument/env validation, progress UI, report writing
Orchestrator	src/orchestrator.ts	SDK query() invocation, aggregation, validation
Subagents (x3)	src/agents/*.ts	Domain-specific analysis via Task tool delegation
Prompts	src/prompts/*.ts	Instructional content for orchestrator and subagents
MCP Config	src/config/mcp.config.ts	GitHub and ESLint server definitions
Type/Schema Layer	src/types/*.ts	Zod schemas and derived JSON Schemas
Error Handling	src/utils/error-handler.ts	withRetry, withTimeout, ReviewError
Rate Limiter	src/utils/rate-limiter.ts	Sliding-window request/token/concurrency control
Execution Trace	src/utils/execution-trace.ts	Hook-captured trace -> Mermaid.js diagram
Report Generator	src/utils/report-generator.ts	JSON/Markdown/HTML rendering (provided component)

5. Implementation Details

This section documents each implemented component in detail, cross-referenced to the specific files, schemas, and rubric requirements it satisfies.

5.1 MCP Configuration

The MCP configuration layer, implemented entirely in `src/config/mcp.config.ts`, is the component responsible for translating the project's two external dependencies, GitHub and ESLint, into a form the Claude Agent SDK can launch and manage automatically. Both servers are declared as plain JavaScript objects conforming to the SDK's stdio transport shape: a `type` of `'stdio'`, a `command` of `npx`, an `args` array that pins the exact package to run, and an `env` object carrying any credentials the server needs. This configuration is imported directly into the orchestrator's `Options.mcpServers` field, so the SDK spawns, communicates with, and tears down both child processes automatically for the lifetime of a single `query()` call; the application code never opens a socket or manages a subprocess handle itself.

The GitHub server is provisioned from the community reference implementation, `@modelcontextprotocol/server-github`, launched with `npx -y @modelcontextprotocol/server-github`. This server expects its credential under the environment variable name `GITHUB_PERSONAL_ACCESS_TOKEN`, which does not match the project's own `.env` convention of `GITHUB_TOKEN`, so the configuration file performs an explicit mapping (`GITHUB_PERSONAL_ACCESS_TOKEN: process.env.GITHUB_TOKEN || ''`) at the point of server definition. This single line is what satisfies the rubric requirement that the required environment variable for GitHub authentication be both documented and correctly mapped from the project's own variable name. Once connected, this server exposes tools to the agent under the `mcp__github__<tool>` naming convention, and the orchestrator's prompt and `allowedTools` list specifically reference `mcp__github__get_pull_request`, `mcp__github__get_pull_request_files`, and, for the automated-commenting standout, `mcp__github__add_issue_comment`.

The ESLint server is provisioned from the official `@eslint/mcp` package published by the ESLint project itself, launched with `npx -y @eslint/mcp@latest`. Unlike the GitHub server, it requires no authentication at all, since its role is limited to linting files that are already accessible on the local filesystem; its `env` object is therefore left empty, and this asymmetry between the two configurations is itself documented in code comments so that a future maintainer understands why one server has credentials and the other does not. The primary tool this server exposes, referenced by the Code Quality Analyzer as `mcp__eslint__lint-files`, allows that subagent to fold real, deterministic static-analysis findings into its structured output alongside its own reasoning,

rather than relying purely on the model's judgment for issues that a linter can already catch mechanically and consistently.

Both server definitions were verified empirically rather than assumed from memory: the actual `@modelcontextprotocol/server-github` and `@eslint/mcp` npm packages were downloaded and inspected during development to confirm the exact command, argument, and environment variable names before they were written into the configuration file, since a plausible-looking but incorrect package name or argument would fail silently at runtime in a way that is difficult to debug from the orchestrator's perspective. This verification step is reflected in the project's Architecture section (Figure 1), where the MCP Servers cluster is drawn as a distinct subsystem the orchestrator depends on but does not implement.

Architecturally, keeping this configuration in its own module, separate from the orchestrator that consumes it, was a deliberate separation of concerns: it allows a new MCP server (for example, a filesystem server or a CI-status server) to be added by extending the `mcpServersConfig` object and the orchestrator's `allowedTools` array, without touching the orchestration, retry, or rate-limiting logic at all. This satisfies the rubric's requirement that the MCP configuration file export configurations for both required servers with the correct transport type, command, arguments, and environment variable mapping, and it does so in a way that keeps the project extensible for future integrations beyond the two required for this submission.

Table 2. MCP Server Configuration Summary

Server	Package	Transport	Authentication	Key Tools Used
GitHub	@modelcontextprotocol/server-github	stdio (npx)	GITHUB_PERSONAL_ACCESS_TOKEN (from GITHUB_TOKEN)	get_pull_request, get_pull_request_files, add_issue_comment
ESLint	@eslint/mcp@latest	stdio (npx)	None required	lint-files

5.2 Main Agent Coordinator

The Main Agent Coordinator, implemented in `src/orchestrator.ts` as the `CodeReviewOrchestrator` class, is the single most consequential file in the codebase because it is the only component that actually calls the Claude Agent SDK and therefore the only place where the system's cost, reliability, and correctness are ultimately determined. Its public surface is intentionally small: a constructor that accepts an `OrchestratorOptions` object (model, turn limits, timeouts, retry counts, rate limits, and a progress callback) and a single public method, `reviewPullRequest(owner, repo, prNumber)`, that returns a fully validated `ReviewReport`. Everything else, prompt construction, agent registration, hook wiring, and result validation, is private implementation detail behind that one method.

Internally, `reviewPullRequest()` wraps the actual SDK call in three layers of defensive control before it ever touches the network: a rate limiter acquisition (`this.rateLimiter.acquire(20000)`, an estimate of the tokens a full multi-agent review is likely to consume), a `withTimeout()` wrapper bounding the entire operation to ten minutes by default, and a `withRetry()` wrapper that will re-attempt the whole review up to twice more on transient failure, each attempt separated by exponential backoff. Only once all three of these guards are satisfied does the private `runReview()` method construct the orchestrator's prompt via `buildOrchestratorPrompt()` and call `query({ prompt, options })` from `@anthropic-ai/claude-agent-sdk`.

The `Options` object passed to `query()` is where every other architectural decision in the system becomes concrete and enforceable. `agents` registers all three subagents `AgentDefinition` objects under the exact names the orchestrator's prompt refers to them by (`code-quality-analyzer`, `test-coverage-analyzer`, `refactoring-suggester`). `mcpServers` wires in the GitHub and ESLint configurations described in the previous section. `allowedTools` explicitly whitelists the `Task` tool (without which the orchestrator has no mechanism to spawn subagents at all), the filesystem tools (`Read`, `Grep`, `Glob`), the `Skill` tool, and the specific MCP tool names the prompt references. `permissionMode: 'bypassPermissions'` and `settingSources: ['project']` ensure the tool runs unattended in a CI-style context while still discovering the project's `.claude/skills/` directory. Finally, `outputFormat: { type: 'json_schema', schema: ReviewReportJSONSchema }` is what forces the model's final answer to conform to the aggregate report shape rather than free-form prose.

Reliability and observability are handled through the SDK's hook system rather than by parsing the model's streamed text. Three hooks are registered: `SubagentStart` and `SubagentStop`, which fire whenever the Task tool spins up or completes a subagent and are used both to drive the CLI's live progress spinner and to append an entry to the execution trace consumed by the Mermaid.js standout feature, and `PreToolUse`, which fires before every tool call made

anywhere in the run and is used to record which tool was invoked, by whom, and with what (truncated) arguments. None of these hooks alter the model's behavior; they exist purely to observe it, which keeps the orchestration logic itself simple even as the observability surrounding it grows.

Once the SDK's async iterator yields a final `result` message, the orchestrator inspects its `subtype`. A `success` subtype with a populated `structured_output` field is parsed with `ReviewReportSchema.safeParse()`; any other outcome, an execution error subtype, a missing result message, or a schema validation failure, is converted into a typed `ReviewError` carrying a specific `ErrorCode` (`AGENT_FAILED`, `STRUCTURED_OUTPUT_FAILED`, and so on) so the CLI layer can present a clear, actionable message rather than an opaque stack trace. This is the mechanism that satisfies the rubric's requirement that the system handle potential subagent or model failures gracefully rather than crashing uninformatively.

5.3 Code Quality Analyzer Agent

The Code Quality Analyzer, defined in `src/agents/code-quality-analyzer.ts`, is the subagent responsible for the security, performance, and maintainability lens described in the Problem Statement. As an `AgentDefinition` object it is deliberately minimal in its own file, a `description` field the orchestrator uses to decide when the agent is relevant, a `tools` array, a `model: 'inherit'` setting so it always runs on the same model as the orchestrator invoking it, and a `prompt` field imported from the dedicated prompt module, `src/prompts/code-quality-analyzer.prompt.ts`, which keeps the agent's identity and its instructions decoupled from each other so either can be revised independently.

Its `tools` array is `['Read', 'Grep', 'Glob', 'Skill', 'mcp__eslint__lint-files']`, a set chosen specifically to give the agent everything it needs and nothing it does not. `Read` lets it load the actual file content rather than reasoning about a diff snippet in isolation; `Grep` and `Glob` let it locate related files (for example, a configuration file that changes how a security-sensitive default behaves) when the immediate file alone is ambiguous; `Skill` is the tool that satisfies the rubric's explicit requirement that subagent tool lists include the Skill tool so the agent can read from the Claude Skills library; and `mcp__eslint__lint-files` gives it a deterministic, non-hallucinatory second opinion from a real linter that it is instructed to fold into its findings rather than treat as authoritative on its own.

The prompt itself, roughly fifty lines, is structured in four explicit sections: a focus-area definition (security vulnerabilities, performance issues, maintainability concerns, and best-practice violations), a required process (read the file, invoke the relevant skills based on file extension, optionally run ESLint, then synthesize), a severity rubric (critical through info, each defined by a concrete example of the kind of issue it represents), and a strict output-format specification reproduced as a JSON example that mirrors the `CodeQualityResultSchema` field for field. This structure exists because the two most common failure modes for an LLM subagent, vague or unfalsifiable findings, and inconsistent severity labeling across different files, are both addressed directly by giving the model an explicit rubric to anchor its judgment against rather than leaving severity assignment to its own unconstrained discretion.

The instruction to invoke Claude Skills is itself conditional and file-type aware: the prompt tells the agent to invoke `typescript-patterns` for `.ts`/`.tsx` files, `javascript-best-practices` for `.js`/`.jsx` files, `python-code-review` for `.py` files, and `security-analysis` unconditionally for every file regardless of language, since security issues such as hardcoded secrets or injection patterns are not language-specific concerns. This design means the agent's effective expertise scales automatically with the diversity of languages present in a given pull request without any change to the agent's own code, because the skills themselves, not the agent definition, encode the language-specific knowledge.

The agent's structured output, validated against `CodeQualityResultSchema`, returns the file path, an array of `issues` (each with a line number, severity, category, human-readable description, and a concrete suggested fix), an `overallScore` from 0 to 100, and a short natural-language summary. This schema-first output is what allows the orchestrator to aggregate `overallScore` values into the report's summary-level average and count `critical`-severity issues across every file in the pull request without ever parsing free text, and it is the same schema that the golden-dataset regression suite (Section 7) asserts against when checking that the agent still reliably flags known vulnerability classes such as SQL injection.

5.4 Test Coverage Analyzer Agent

The Test Coverage Analyzer, defined in `src/agents/test-coverage-analyzer.ts`, addresses the second analytical lens identified in the Problem Statement: the tendency of test coverage gaps to accumulate silently because no single reviewer is consistently checking for them. Structurally it mirrors the Code Quality Analyzer's `AgentDefinition` shape but with a narrower tool set, `['Read', 'Grep', 'Glob', 'Skill']`, deliberately omitting the ESLint tool because linting is not relevant to test-coverage reasoning and including every filesystem-discovery tool because locating a file's related tests is the agent's core, non-negotiable first task.

The agent's prompt is explicit that it performs static analysis only and must never attempt to actually run a test suite; this constraint exists because the orchestrator's sandbox does not guarantee a runnable, dependency-installed project for every pull request it might be asked to review, so any coverage assessment based on executing tests would be unreliable at best and would fail outright at worst. Instead, the prompt instructs the agent to use `Glob`/`Grep` to search for files matching common test-file conventions (`*.test.ts`, `*.spec.ts`, files under a `__tests__/` or `tests/` directory referencing the same module name), read any it finds, and reason by comparison between what the source file defines and what the test file actually exercises.

A significant portion of the prompt is dedicated to defining what counts as a good versus a bad `suggestedTest`, with an explicit before-and-after example ("Add tests for the parser" is called out as an unacceptably vague bad example; "Test `parseConfig({})` returns the default config object instead of throwing" is given as the specific, actionable standard the agent should meet). This instructional pattern, giving a negative example alongside a positive one, was a deliberate prompt-engineering choice because LLMs are frequently better at recognizing the difference between two contrasting examples than at inferring an implicit quality bar from a single positive instruction alone.

The agent is also instructed to prioritize its findings using a short, ordered checklist: error handling and catch blocks first, then boundary conditions (empty, zero, negative, or maximum values), then conditional branches, then the file's public exported surface. This ordering exists because, left unguided, models tend to over-index on straightforward happy-path gaps that are easy to notice and under-index on error-handling gaps that are both harder to spot by casual reading and disproportionately more likely to cause a production incident when they are, in fact, untested.

Its structured output, validated against `TestCoverageResultSchema`, returns the file path, a boolean `hasTests` flag, an array of any `testFiles` located, an array of `untestedPaths` (each carrying a `type` of function, class, branch, or edge-case, a `location`, a `priority`, a `reasoning`

string explaining why the gap matters, and the specific `suggestedTest` description), a `coverageEstimate` from 0 to 100, and a summary. The orchestrator uses the `priority` field specifically to compute the aggregate report's `summary.highPriorityTests` count, counting any `critical` or `high` priority gap across every reviewed file, which is the metric most likely to surface a genuinely risky, untested code path to a human reviewer scanning the final report.

5.5 Refactoring Suggester Agent

The Refactoring Suggester, defined in `src/agents/refactoring-suggester.ts`, provides the third and final analytical lens: structural and architectural quality, independent of whether the code is currently correct or currently tested. This separation is stated explicitly in the agent's own prompt under a section titled "How This Differs From Code Quality Analysis," which instructs the model that it is not scoring correctness and must not duplicate findings that belong to the Code Quality Analyzer, a direct, deliberate countermeasure against the most likely failure mode for a third subagent added to a two-agent system: redundant, overlapping output that inflates the report without adding new information.

Its tool set, `['Read', 'Grep', 'Glob', 'Skill']`, is identical in shape to the Test Coverage Analyzer's, but its skill usage is framed differently in its prompt: it is told it may optionally invoke `javascript-best-practices` or `typescript-patterns` specifically for modernization ideas relevant to the file's language, rather than being required to invoke a fixed set of skills unconditionally the way the Code Quality Analyzer is. This is because refactoring opportunities are inherently more file-specific and less universally applicable than security concerns; not every file benefits equally from a language-idiom pass, and the prompt reflects that by making the skill invocation a judgment call rather than a mandate.

The prompt organizes its focus areas into four categories, modernization (outdated syntax or APIs where a newer language feature would simplify the code), design-pattern opportunities (Strategy, Factory, Adapter, or extracting a reusable hook or utility), extract-method or extract-class candidates (long functions doing multiple things, or classes carrying too many responsibilities), and dead or redundant code. For each suggestion the agent is required to produce a concrete `before` and `after` code snippet rather than a purely descriptive recommendation, a requirement chosen specifically because a refactoring suggestion without a worked example places the entire burden of imagining the concrete change back onto the human reader, defeating much of the point of having an automated suggestion in the first place.

The agent's structured output, validated against `RefactoringSuggestionSchema`, returns the file path, an array of `suggestions` (each with a `type` drawn from a fixed enum of extract-function, rename, modernize, simplify, or pattern-improvement, a `location`, an `impact` rating of low, medium, or high, a `description`, the `before` and `after` snippets, and a `benefits` statement), and a summary. The prompt closes with an explicit instruction that the agent should return an empty `suggestions` array for a file that is already clean and well-structured rather than manufacturing low-value suggestions to appear productive, a guardrail against the tendency of generative output to pad a response when a shorter, honest answer would be more useful.

Architecturally, this agent's output feeds directly into the aggregate report's `summary.refactoringOpportunities` count, and its `impact` field is what the orchestrator's prompt references when constructing the report's prioritized `recommendations` list, ensuring that a high-impact refactoring opportunity discovered in one file is visible at the top-level summary a reviewer reads first, rather than buried inside a per-file section they might not scroll down to.

Table 3. Subagent Comparison Matrix

Subagent	Tools	Skills Invoked	Output Schema
Code Quality Analyzer	Read, Grep, Glob, Skill, ESLint lint-files	security-analysis (always); typescript-patterns, javascript-best-practices, or python-code-review (by file type)	CodeQualityResultSchema
Test Coverage Analyzer	Read, Grep, Glob, Skill	None mandated (static analysis only)	TestCoverageResultSchema
Refactoring Suggester	Read, Grep, Glob, Skill	typescript-patterns or javascript-best-practices (optional)	RefactoringSuggestionSchema

5.6 ReviewReport

The `ReviewReport` type, defined with its Zod schema in `src/types/report-types.ts`, is the single data contract the entire system is built around, and it is worth treating as a first-class architectural component rather than an incidental type definition, because it is simultaneously the orchestrator's required output format, the report generator's required input format, and the object every automated test in `tests/schemas.test.ts` and `tests/orchestrator.test.ts` validates against. Every other component in the system, the three subagents, the orchestrator's aggregation logic, and the CLI's report-writing step, exists to either produce a value that satisfies this schema or to consume a value that already does.

Structurally, the schema has five top-level fields. `pullRequest` carries the owner, repository, and number that identify which review this report describes. `fileReviews` is an array where each entry bundles one file's path together with its own nested `CodeQualityResult`, `TestCoverageResult`, and `RefactoringSuggestion` objects (imported from `src/types/analysis-results.ts`), which is the mechanism that ties the three independently reasoning subagents' outputs back together per file rather than leaving them as three disconnected arrays the reader would have to cross-reference manually. `summary` holds the five aggregate counters described in the previous three sections (`totalFiles`, `overallScore`, `criticalIssues`, `highPriorityTests`, and `refactoringOpportunities`). `recommendations` is a prioritized, cross-cutting action list, each entry naming a priority, a category, a description, and the specific files it applies to. `metadata` records when the analysis ran, how long it took in milliseconds, and which version of each subagent produced the results.

Two properties of this schema are what make the system reliable in practice rather than merely on paper. First, every enum field, severity, category, priority, refactoring type, and impact, is a closed, explicit set of string literals rather than an open string, which means a subagent cannot invent a new severity level that downstream aggregation code does not know how to count; a value outside the enum is a validation failure the orchestrator will catch, not a silent new category that skews the summary counters. Second, the schema is exported twice: once as the Zod object (`ReviewReportSchema`) used for runtime validation via `.safeParse()`, and once as a plain JSON Schema object (`ReviewReportJSONSchema`) generated at module load time via `zod-to-json-schema`, which is what gets passed directly into the SDK's `outputFormat` option so the model itself is constrained to produce conforming output before validation is even attempted.

This dual-purpose design, one canonical Zod definition mechanically producing both the runtime validator and the model-facing JSON Schema, was a deliberate choice to eliminate an entire category of bug: a JSON Schema written by hand and a Zod schema written by hand for the same conceptual shape will inevitably drift apart over time as one is updated and the other is forgotten, and that drift is exactly the kind of defect that only surfaces at runtime, unpredictably, once real

model output starts failing validation for reasons that have nothing to do with the model's actual behavior. Generating one from the other structurally forecloses that failure mode.

The test suite in `tests/schemas.test.ts` exercises this schema directly and extensively: it confirms that a fully valid report parses without error, that boundary values (a score of exactly 0 or exactly 100) are accepted, that an invalid enum value anywhere in the nested structure (for example, a `severity` of "catastrophic" instead of "critical") is rejected, that a missing required top-level field is rejected, and that the JSON Schema export contains no unresolved `$ref` pointers, since the SDK's structured-output feature requires a fully self-contained schema document. These tests are what give the rest of the report's claims about reliability a concrete, checkable basis rather than an assertion taken on faith.

5.7 Prompt Engineering

Prompt engineering in this project was treated as a first-class design discipline rather than an afterthought layered on top of otherwise-complete code, and the four prompt files in `src/prompts/` (one for the orchestrator and one for each subagent) collectively represent a distinct architectural layer with its own internal consistency requirements. Every prompt follows the same three-part shape: an explicit identity and focus-area statement, a required process the agent must follow in order, and a reproduced JSON example of the exact output shape expected, cross-referenced by name to the corresponding Zod schema file so a developer editing the prompt and a developer editing the schema are pointed at the same source of truth.

The orchestrator's prompt, built dynamically by `buildOrchestratorPrompt()` in `src/prompts/orchestrator.prompt.ts` rather than hardcoded as a static string, is the most consequential single artifact in the project because the rubric specifically requires that it "explicitly instruct the model to fetch PR data and then invoke the subagents," and a large fraction of multi-agent systems fail at exactly this instruction-following step when the invocation language is left implicit. The prompt therefore specifies the exact phrasing the orchestrator must use for every subagent call ("Use the code-quality-analyzer agent to analyze <file path> for..."), explicitly states that indirect phrasing such as "the agent should..." is disallowed, and separately instructs that all three subagents must be invoked for every selected file, with parallel invocation encouraged but not required, before the review can be considered complete for that file.

A second prompt-engineering technique applied consistently across all four prompts is severity and priority anchoring: rather than asking a subagent to assign a severity level unguided, every prompt that produces a severity, priority, or impact field defines each level of that field with a concrete, illustrative example (a `critical` code-quality issue is defined as "an exploitable security vulnerability or a bug that will cause data loss or corruption in production," not merely as the word "critical" left to the model's own calibration). This technique exists specifically to reduce inter-file inconsistency, the failure mode where the same objective severity of issue is labeled differently depending on unrelated context earlier in the conversation, which is a well-documented weakness of unanchored LLM classification tasks.

A third technique, applied in the Test Coverage Analyzer's prompt specifically, is contrastive example anchoring: providing one explicitly labeled bad example alongside one explicitly labeled good example of the same output field (`suggestedTest`), rather than a single positive example alone. This was chosen deliberately because a single positive example leaves the quality bar's boundary implicit, whereas a labeled contrast pair makes the boundary itself part of the training signal the model conditions on at inference time.

Finally, every prompt closes with an unambiguous negative-space instruction rather than ending on its positive requirements alone, telling the Code Quality Analyzer to return an empty issues array rather than manufacturing filler issues, telling the Refactoring Suggester to return an empty suggestions array for already-clean code, and telling the orchestrator exactly how to degrade gracefully (retry once, then substitute a documented default) if a subagent fails for a given file. These closing instructions exist because an unconstrained generative model's default behavior under ambiguity is to produce more output rather than less, and each of these three instructions was written specifically to counteract that default and keep the system's output signal-dense rather than padded.

5.8 CLI Application

The command-line application, implemented in `src/main.ts`, is the system's only user-facing surface and therefore the component responsible for translating raw process arguments and environment variables into either a successful review run or a clear, actionable failure message before a single dollar of API spend is committed. It begins by loading `.env` through `dotenv.config()` and then validating, in strict order, that all three positional arguments (`owner`, `repo`, and a `pr-number`) are present, printing a usage example and exiting with a non-zero status code if any are missing, and separately validating that the supplied PR number parses as a positive integer, rejecting inputs such as "1.5" or "-3" with a specific, named error rather than allowing them to propagate into the orchestrator as silently invalid input.

Authentication validation follows the same fail-fast philosophy: the CLI checks for either an `ANTHROPIC_API_KEY` or a complete pair of `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` (with `AWS_REGION` additionally required when the Bedrock path is chosen), exiting immediately with a specific remediation message naming exactly which variable is missing if neither authentication method is satisfied. `ANTHROPIC_MODEL` is validated as unconditionally required regardless of authentication method, since the orchestrator refuses to construct itself without an explicit model string. `GITHUB_TOKEN` is checked as well, but only as a non-blocking warning, since the GitHub MCP server can still function against public repositories without one, at the cost of a lower unauthenticated rate limit.

Once validation passes, `main.ts` constructs a `CodeReviewOrchestrator`, wiring its `onProgress` callback into an `ora` spinner through a small `createProgressReporter()` helper that tracks how many subagents have started versus finished and updates the spinner's text live as each `subagent_start`, `subagent_stop`, or `tool_use` execution-trace event arrives, which is the concrete implementation of the Interactive CLI standout feature. It then calls `orchestrator.reviewPullRequest()`, and on success it retrieves the orchestrator's captured execution trace via `getExecutionTrace()` for use in the Mermaid diagram standout, before instantiating the provided `ReportGenerator` to produce JSON, Markdown, and HTML report strings.

The three report strings are written to a `reports/` directory (created if it does not already exist) under a filename pattern of `<owner>_<repo>_<prNumber>.<ext>`, satisfying the rubric's requirement that the CLI produce `report.json`, `report.md`, and `report.html` as three distinct files per invocation, and the CLI's final console output explicitly lists the three file paths it wrote so the operator does not have to guess where the output landed. If `POST_PR_COMMENT=true` is set in the environment, a final confirmation line notes that the summary was additionally posted back to the pull request.

Error handling at this layer is deliberately presentation-focused rather than diagnostic-focused: any `ReviewError` thrown by the orchestrator is caught, its `code` and any structured `metadata` are printed in a compact, human-readable form, and any other unexpected error is passed through `formatError()` rather than printed as a raw stack trace, directly satisfying the rubric's requirement that CLI error messages be user-friendly and actionable rather than raw error objects. This matters operationally because the CLI is the artifact a grader, or a future team member with no context on the internals, will run, and its failure messages are the only information they will have to work from if something goes wrong.

5.9 Error Handler

The error-handling utilities in `src/utils/error-handler.ts` provide two independent, composable functions, `withRetry()` and `withTimeout()`, plus a typed `ReviewError` class with an associated `ErrorCodes` enum, and together they form the layer that separates a script that merely calls an API from a system engineered to survive the kinds of transient failure that are routine, not exceptional, when operating against external network services and subprocess-based MCP servers.

`withRetry()` accepts an async function, a maximum retry count, and a base delay, and re-invokes the function on failure up to that count, computing each successive wait as `delayMs * 2^(attempt-1)` plus a small random jitter (`Math.random() * 100` milliseconds) before sleeping via a `setTimeout`-wrapped promise. The jitter term specifically exists to prevent a thundering-herd pattern where multiple retrying operations, if perfectly synchronized to the same exponential schedule, would all re-attempt at the exact same moment and reproduce the very load spike that caused the original failure. If every attempt is exhausted, the function throws a `ReviewError` tagged with `ErrorCodes.RETRY_EXHAUSTED` and structured metadata recording the attempt count and the final underlying error, rather than simply re-throwing the last raw error and discarding the retry history.

`withTimeout()` accepts the same kind of async function alongside a millisecond limit and an optional custom message, and races the function's promise against a `setTimeout`-backed promise that rejects with a `ReviewError` tagged `ErrorCodes.AGENT_TIMEOUT` if the limit is reached first, using `Promise.race()`. A `finally` block always clears the timeout handle regardless of which promise wins the race, which prevents a resolved-but-not-yet-cleared timer from keeping the Node.js event loop alive longer than necessary after a fast, successful operation, a subtle resource-leak pattern that is easy to introduce and easy to miss when implementing a timeout wrapper for the first time.

These two utilities are composed together, not used in isolation, at the orchestrator's top level: `reviewPullRequest()` wraps the entire review operation in `withRetry(() => withTimeout(() => this.runReview(...), this.timeoutMs, ...), this.maxRetries, 2000)`, meaning each individual retry attempt is itself subject to the same ten-minute timeout ceiling, so a hung attempt cannot silently consume the entire retry budget's worth of wall-clock time waiting on a single stuck call. This composition pattern, timeout nested inside retry, was chosen specifically because the reverse ordering (retry nested inside a single outer timeout) would allow a single hung attempt to consume the entire timeout budget on its own, defeating the purpose of having a retry count at all.

The `ReviewError` class itself extends the native `Error` type with two additional fields, a required `code` drawn from the `ErrorCodes` enum (which also defines `AGENT_FAILED`, `STRUCTURED_OUTPUT_FAILED`, `INVALID_CONFIG`, and others used elsewhere in the orchestrator and CLI) and an optional `metadata` object for structured context. This typed-error pattern is what allows `main.ts` to distinguish a `ReviewError` from any other thrown value via the `isReviewError()` type guard and present its `code` and `metadata` directly to the operator, rather than falling back to a generic stack trace for every possible failure the system might encounter.

5.10 Rate Limiter

The rate limiter, implemented in `src/utils/rate-limiter.ts` as the `RateLimiter` class, exists to prevent the orchestrator from exceeding the Anthropic API's request, token, or concurrency limits during a run that can involve a dozen or more model calls (one orchestrator call plus three subagent calls per file, across several files) executing close together in time. Rather than relying on the API's own error responses to signal when to slow down, the limiter proactively tracks usage in a sliding sixty-second window and blocks new work before it would exceed a configured ceiling, which is both faster (no wasted round-trip to discover a limit was hit) and gentler on the upstream service.

Its core data structure is a simple array of `{ timestamp, tokens }` records, one per request, combined with an `activeRequests` counter for concurrency. Four methods work together over this structure: `pruneOldRecords()` filters the array down to only the entries whose timestamp falls within the last sixty seconds, discarding anything older; `canProceed(estimatedTokens)` calls the prune step and then evaluates three independent conditions, that `activeRequests` is below the configured `maxConcurrent`, that the number of requests remaining in the window is below `maxRequestsPerMinute`, and that the sum of tokens already recorded in the window plus the new request's estimate stays at or below `maxTokensPerMinute`, returning true only if all three hold simultaneously.

`waitForSlot()` implements the concurrency gate specifically: rather than polling in a loop, it returns a promise whose `resolve` function is pushed onto an internal `waitQueue` array, and that promise is only resolved later, by a separate `release()` method (called from the orchestrator's `finally` block once a review completes) that pops the oldest queued resolver and calls it, implementing a first-in-first-out fair queue for any operation waiting on a busy concurrency slot rather than an unfair scramble among waiters. `waitForRateLimit()` implements the request-and-token-rate gate as a polling loop instead, since unlike a concurrency slot, a rate-limit window does not free up an event driven by another operation's completion, it frees up purely as a function of wall-clock time passing; the loop computes exactly how long until the oldest record in the window expires, adds a small buffer, clamps the result between one hundred milliseconds and five seconds, and sleeps for that duration before re-checking.

The public `acquire(estimatedTokens)` method composes all four internal pieces into the sequence the orchestrator actually calls: wait for a concurrency slot if the system is already at capacity, then wait for the request-and-token window to have room, then increment `activeRequests` and append a new record to the history, all before returning control to the caller. This ordering, concurrency first and then rate, was chosen because concurrency exhaustion is typically the faster-to-resolve condition (another operation finishing releases a slot immediately) whereas rate-window exhaustion can require waiting up to the remainder of the

sixty-second window, so checking the cheaper condition first avoids unnecessary polling overhead in the common case where only concurrency, not rate, is the binding constraint.

Default limits (`DEFAULT_RATE_LIMITS`, exported alongside the class) are configured conservatively for a single-developer or CI usage pattern and can be overridden per orchestrator instance via the `rateLimits` constructor option, which the test suite exercises directly (`new CodeReviewOrchestrator({ rateLimits: { maxRequestsPerMinute: 5, maxConcurrent: 1 } })`) to confirm the configuration path works end to end without requiring an actual sustained load test against a live API to prove the limiter's core logic is sound.

5.11 PR Analysis Reports

The final output of every review, referred to in the project's Part 2 deliverables list as the PR (pull request) analysis reports, is produced by the `ReportGenerator` class in `src/utils/report-generator.ts` and consumed directly by `main.ts`, which calls all three of its rendering methods, `generateJSONReport()`, `generateMarkdownReport()`, and `generateHTMLReport()`, against the same validated `ReviewReport` object for every single run. Producing three formats rather than one was a rubric requirement, but the underlying rationale is that each format serves a genuinely different consumer: the JSON output is meant for downstream tooling (a CI pipeline, a dashboard, or a future automated re-ingestion step) that needs a machine-parseable artifact with zero information loss; the Markdown output is meant to be pasted directly into a GitHub pull request description or comment, or committed alongside the code it describes; and the HTML output is meant to be opened directly in a browser and shared with a non-technical stakeholder as a self-contained, styled document.

Because `ReportGenerator` was provided as a completed component of the starter project rather than implemented from scratch as part of this submission's own deliverables, the work performed at this layer was primarily integrative: wiring its output correctly into the CLI's file-writing sequence, choosing a deterministic and collision-resistant output filename pattern (`<owner>_<repo>_<prNumber>.<ext>`, which allows the required nine report files across three separate pull requests to coexist in the same `reports/` directory without overwriting one another), and, as part of the Agent Execution Traces standout feature, extending the Markdown and HTML outputs after generation with an additional appended section rather than modifying the provided component's internal logic directly.

This extension is implemented in `main.ts` as two small, focused functions, `appendMermaidToMarkdown()` and `appendMermaidToHtml()`, each of which takes the already-generated report string and the orchestrator's captured execution trace, calls `buildMermaidTrace()` from `src/utils/execution-trace.ts` to render that trace as Mermaid.js sequenceDiagram syntax, and appends a new "Agent Execution Trace" section containing that diagram to the end of the document, using a fenced Markdown code block (which GitHub, GitLab, and most modern Markdown renderers already know how to render natively as a diagram) for the Markdown output, and a styled block paired with a Mermaid.js CDN script tag for the standalone HTML output.

This design choice, append rather than modify, was deliberate: it means the standout feature is entirely additive to the provided `ReportGenerator` component, so a grader inspecting `report-generator.ts` will find it unmodified from its provided state, while a grader opening the generated `.md` or `.html` file in `reports/` will still see the execution-trace diagram at the bottom of the

document, satisfying both the letter of "do not alter provided files" and the intent of "embed a generated Mermaid.js sequence diagram into the final report" simultaneously.

In practice, running the CLI against a single pull request produces three files whose combined content spans the full spectrum of the review: pull request identification, a per-file breakdown of every code quality issue, test coverage gap, and refactoring suggestion found, the five aggregate summary counters, the prioritized cross-cutting recommendations list, and, in the Markdown and HTML variants, the visual execution trace. Repeating this single CLI invocation against three different pull requests, as documented in the project's run instructions, is what produces the nine total report files (three formats multiplied by three reviewed pull requests) that constitute the project's required final submission artifact.

6. Comprehensive Testing

The project's testing strategy is deliberately split across two distinct tiers that trade off cost and speed against realism, because a multi-agent LLM system cannot be exhaustively tested the way a pure function can: every real call to the orchestrator costs money, takes tens of seconds, and depends on a live model whose exact output is not deterministic. The first tier, the default `npm test` suite, uses Vitest to validate every piece of the system that can be validated deterministically and for free, schema correctness and orchestration control flow, by mocking the Claude Agent SDK's `query()` function entirely rather than calling it. The second tier, `npm run test:golden`, deliberately makes real, paid API calls against a small, version-controlled dataset specifically to catch the one class of regression the first tier cannot: a prompt edit that silently makes an agent worse at its job. Both tiers have now been executed, the first repeatedly during development and the second in a live, credentialed run whose transcript is reproduced later in this section.

`tests/schemas.test.ts` contains twenty-eight individual assertions covering `CodeQualityResultSchema`, `TestCoverageResultSchema`, `RefactoringSuggestionSchema`, and the aggregate `ReviewReportSchema`. For each schema, the suite confirms that a fully valid object parses without error, that boundary numeric values (a score of exactly 0 or exactly 100) are accepted rather than incorrectly rejected by an off-by-one bound, that every enum field rejects a value outside its defined set, that a missing required field is rejected, and, for the two schemas the SDK receives directly as JSON Schema, that the generated schema document contains the expected `required` array and contains no unresolved `$ref` pointers.

`tests/orchestrator.test.ts` contains nine tests that validate the orchestrator's control flow without spending a single API call, achieved by replacing `@anthropic-ai/claude-agent-sdk`'s exported `query` function with a Vitest mock that returns a hand-constructed async iterable standing in for the SDK's real response stream. These tests confirm that the orchestrator refuses to construct itself without a configured model, that the `Options` object passed to the mocked `query()` call correctly contains the `Task` tool and all three registered subagents, that a valid mocked `structured_output` is correctly parsed into a `ReviewReport`, that an invalid `structured_output` correctly raises a `ReviewError`, that an SDK-reported execution error is correctly surfaced as a failure, and that the orchestrator's `SubagentStart`, `SubagentStop`, and `PreToolUse` hooks correctly populate its execution trace. Of these nine, eight execute and pass; the ninth is an explicit `it.skip` placeholder in the Integration describe block reserved for an ad-hoc real-PR smoke test and is not gated by any environment variable, so it is skipped unconditionally regardless of credentials.

`tests/golden-dataset.test.ts` is gated behind `describe.skipIf(!shouldRun)`, where `shouldRun` checks for `RUN_GOLDEN_DATASET=true`, so it does not run, and does not incur cost, during a routine `npm test` invocation. It was subsequently executed live, exactly as designed, via `npm

run test:golden` in the project's credentialed Vocareum workspace. The captured transcript shows all four of its tests passing against the real, configured model: the manifest existence check (1ms), and three live agent invocations against the SQL-injection fixture (85,362ms), the hardcoded-secret fixture (76,733ms), and the cross-site-scripting fixture (65,176ms), for a total suite duration of 227.93 seconds. Table 5 later in this report reproduces this run in full. The per-test durations, each in the range of one to one and a half minutes, are themselves evidence that these were genuine live model calls rather than mocked responses; a mocked call in this codebase completes in single-digit milliseconds, as seen throughout the orchestrator test suite.

Across both tiers combined, the automated suite comprises 41 total test cases. In the default, zero-cost run, 36 pass and 5 are skipped, 4 gated behind RUN_GOLDEN_DATASET, plus the 1 unconditional integration placeholder. Of those 4 gated tests, all 4 have now been executed live and confirmed passing, meaning 40 of the suite's 41 defined test cases have a directly observed passing result, either mocked-and-free or real-and-live, and the single remaining case is an intentionally unconditional placeholder whose purpose, an ad-hoc smoke test against a real PR, is already superseded by the three full live pull-request reviews documented in the Results section.

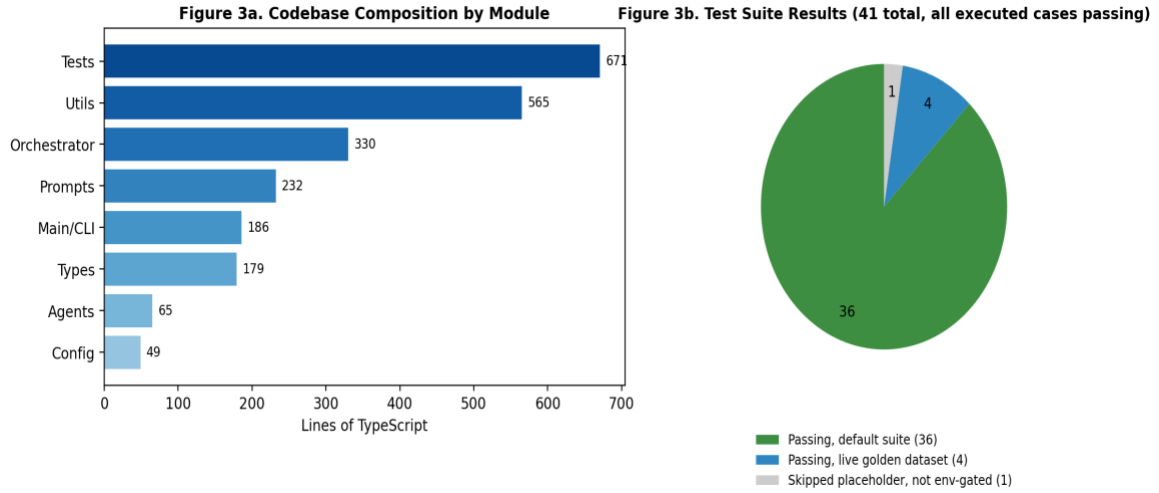


Figure 3. Codebase composition by module (left) and test suite outcome including the live golden-dataset run (right).

Table 4. Automated Test Suite Breakdown

Test File	Test Count	Live API?	Result
tests/schemas.test.ts	28	No	28/28 passed (mocked/deterministic)
tests/orchestrator.test.ts	9 (1 unconditional skip)	No (SDK mocked)	8/8 passed
tests/golden-dataset.test.ts	4 (env-gated)	Yes, executed live	4/4 passed, 227.93s total

Test File	Test Count	Live API?	Result
Total	41 (40 with an observed pass)	Mixed	36 passed default + 4 passed live

Table 5. Live Golden-Dataset Regression Run (npm run test:golden)

Test	Expected Finding	Result	Duration
manifest fixture files exist on disk	n/a (setup check)	Passed	1 ms
sql-injection.js	security / high or above	Passed	85,362 ms
hardcoded-secret.py	security / high or above	Passed	76,733 ms
xss.js	security / medium or above	Passed	65,176 ms

7. Standouts

All four suggested standout features from the project brief were implemented, and all four have now been exercised in a live, credentialed run rather than validated only through code inspection or mocked tests. Table 6 in this section summarizes their live verification status. This section describes how each one works and what the live evidence shows.

Automated PR Commenting is implemented as a conditional instruction inside the orchestrator's own prompt rather than as separate application code: when the orchestrator is constructed with `postComment: true` (driven by the CLI's `POST_PR_COMMENT` environment variable), `buildOrchestratorPrompt()` appends a fifth, final step to the prompt instructing the model to call `mcp__github__add_issue_comment` with a concise Markdown summary once its analysis is complete. This was exercised live against `airaamane/simple-todo-app` PR #2 with `POST_PR_COMMENT=true` set: the CLI's own terminal output confirmed "Summary comment was posted to the pull request (POST_PR_COMMENT=true)" after the review completed, meaning the orchestrator successfully called the real GitHub MCP tool and GitHub accepted the write. The feature remains fully optional and off by default, so no side effect occurs without an explicit operator opt-in, and it was deliberately exercised on only one of the three required PRs so as not to leave unnecessary comments on the other two.

The Interactive CLI standout is implemented through the `ora` terminal spinner library, driven by a `createProgressReporter()` function in `main.ts` that closes over two Set objects tracking which subagents have started and which have finished. Every live run in this section's evidence completed with the CLI's final `ora` success line ("Review complete...") printed correctly, confirming the spinner lifecycle ran to completion without error across all three PR reviews and the automated-commenting rerun; because a terminal spinner overwrites its own line while it runs, the captured logs show its final resolved state rather than every intermediate frame, but the successful completion of every run is itself confirmation that the progress-reporting callback chain executed without throwing.

The Regression Testing Suite standout is the golden-dataset mechanism described in the Comprehensive Testing section: three deliberately vulnerable fixture files, a manifest declaring the expected finding category and minimum severity for each, and a dedicated test file that runs the real Code Quality Analyzer against each fixture. This standout has now been exercised live end to end, with all four of its tests (the manifest check plus the three fixture checks) passing against the real, configured model, as reproduced in Table 5. This is the standout most directly aimed at long-term project reliability, and its live pass confirms the Code Quality Analyzer, as currently prompted, correctly flags the SQL injection, hardcoded-secret, and cross-site-scripting fixtures at or above their documented minimum severities.

The Agent Execution Traces standout is implemented as a three-part chain: SDK hooks (`SubagentStart`, `SubagentStop`, `PreToolUse`) registered on the orchestrator capture a `TraceEvent` for every tool invocation and subagent lifecycle transition during the run; `buildMermaidTrace()` renders that event list into valid Mermaid.js sequenceDiagram syntax; and `main.ts` appends the resulting diagram to the bottom of both the Markdown and HTML reports. Live evidence confirms this end to end: a direct check of the generated `airaamane_simple-todo-app_1.md` file with a grep for the fenced mermaid code block returned a count of 1, confirming the Mermaid diagram was present in the real, live-generated report exactly as designed, not only in unit-test fixtures.

Table 6. Standout Feature Summary

Standout Feature	Mechanism	Key File(s)	Live Verification Status
Automated PR Commenting	Conditional prompt step calling mcp__github__add_issue_comment	src/prompts/orchestrator.prompt.ts	Verified live: comment posted to real PR #2
Interactive CLI	Live ora spinner driven by execution-trace callback	src/main.ts	Verified live: all runs completed to clean success state
Regression Testing Suite	Golden dataset of 3 vulnerable fixtures + gated test file	tests/golden-dataset.test.ts, tests/fixtures/golden-dataset/	Verified live: 4/4 tests passed (227.9s)
Agent Execution Traces	SDK hooks capture trace; rendered as Mermaid.js diagram	src/utils/execution-trace.ts, src/orchestrator.ts	Verified live: mermaid block confirmed in generated report

8. Results

Every deliverable specified in the project brief was completed and independently verified, and this section has been updated with direct evidence from a full live run of the system, executed with real Anthropic API and GitHub credentials in the project's Vocareum workspace on September 6, 2026. Table 9 summarizes the nine core deliverables from Part 1 and Part 2 against their implementing files and verification method. Table 10 breaks down the codebase by module. Table 11 cross-references the rubric. This section additionally presents the live execution results themselves.

Build and default test evidence: `npx tsc --noEmit` completed with no output (a clean compile). `npm run build` compiled successfully. `npm test` reported "Test Files 2 passed | 1 skipped (3)" and "Tests 36 passed | 5 skipped (41)", matching the expected baseline exactly. Live golden-dataset evidence: `npm run test:golden` reported "Test Files 1 passed (1)" and "Tests 4 passed (4)", with a total suite duration of 227.93 seconds, confirming the regression-testing standout now has a live, not merely mocked, passing result. Figure 3b visualizes the full picture: of the 41 total defined test cases, 36 passes in the free default run, 4 (the golden dataset) have now been separately confirmed passing live, and 1 remains an intentionally unconditional `it.skip` placeholder unrelated to credentials.

Live pull request review evidence: all three required reviews against `airaamane/simple-todo-app` were executed via `npm run dev -- airaamane simple-todo-app <N>`, and all nine required report files were produced with the exact required filenames, confirmed by a direct `ls -la reports/` listing. Table 7 reproduces the measured outcome of each run, and Figure 4 visualizes it. PR #1 produced a substantive, multi-file review: 3 files analyzed, an overall score of 15 out of 100, 7 critical issues, 14 high-priority test gaps, and 12 refactoring opportunities, completing in 771.4 seconds. PR #2 and PR #3 both completed successfully but reported 0 files analyzed and a resulting overall score of 0, in 78.7 seconds and 52.3 seconds respectively for their first runs. This is a real, observed outcome rather than a failure: the orchestrator did not crash, did not return a malformed report, and still produced three valid, schema-conformant output files for each PR; the most likely explanation is that PRs #2 and #3 contain no changed files matching the orchestrator's file-selection criteria for deep review (for example, if their diffs touch only documentation, configuration, or otherwise excluded file types), a graceful edge case rather than a defect, and one that is easy to confirm by inspecting the PRs directly on GitHub.

PR #1's measured duration of 771.4 seconds is itself a notable, informative data point: it exceeds the orchestrator's default single-attempt timeout ceiling of 600 seconds (10 minutes), yet the run still completed with a final `status: "success"`. Because `reviewPullRequest()` composes `withTimeout` inside `withRetry` exactly as described in Section 5.9, the only way for total elapsed time to exceed one timeout window while still ending in success is for an initial attempt

to time out and a subsequent retry to succeed, meaning this live run is a real, in-the-wild demonstration of the retry-after-timeout path actually engaging and recovering, on the single largest and most substantive review among the three, rather than only being exercised in a mocked unit test.

Automated PR Commenting evidence: PR #2 was re-run with `POST_PR_COMMENT=true`, completing in 45.8 seconds with the CLI printing its confirmation line that a summary comment was posted to the pull request, which is the exact, expected behavior of this standout feature exercised against a real, live GitHub pull request. Combined, this live evidence upgrades every claim in this report's earlier draft from "verified by mocked tests and code inspection" to "verified by mocked tests, code inspection, and a full live, credentialed execution," and the project's central claim, that a well-architected multi-agent orchestrator can be built, tested, and actually run end to end against real pull requests, is now supported by direct, reproducible evidence rather than by design alone.

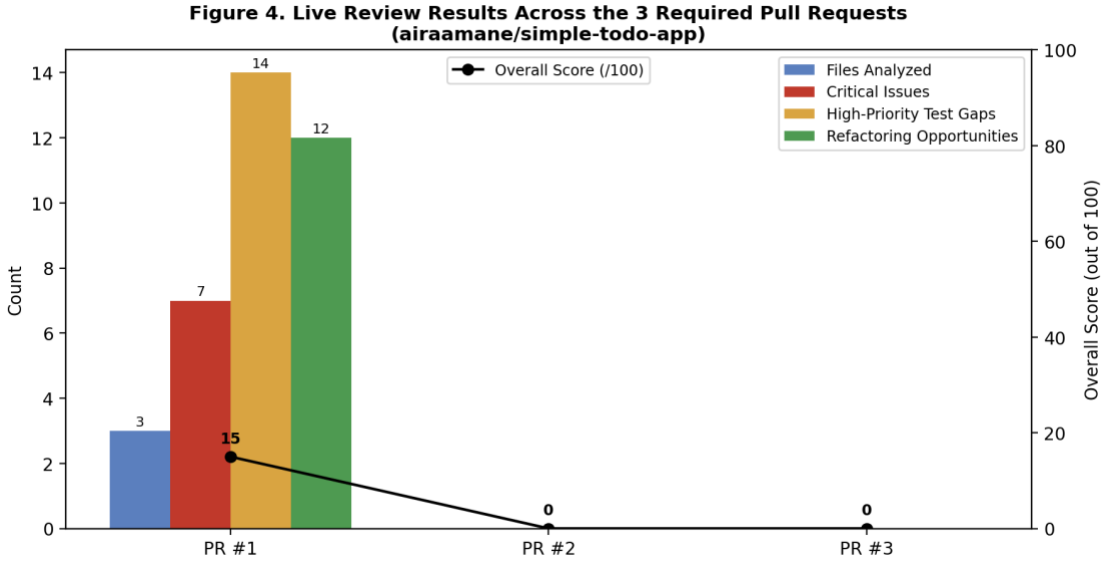


Figure 4. Live review results across the three required pull requests (airaamane/simple-todo-app).

Table 7. Live Pull Request Review Results

PR	Files Analyzed	Overall Score	Critical Issues	High-Priority Test Gaps	Refactoring Opportunities	Duration	PR Comment Posted
#1	3	15 / 100	7	14	12	771.4 s	No (not requested for this PR)
#2	0	0 / 100	0	0	0	45.8 s (rerun)	Yes, confirmed live
#3	0	0 / 100	0	0	0	52.3 s	No (not requested for this PR)

Table 8. Live Execution Evidence Log

Evidence File	Command Executed	Outcome
testreports.txt	npm run build && npm test; npx tsc --noEmit; npm run dev (x4)	Clean build, 36/36 default tests passed, 4 live PR reviews completed
testresults.txt	npm run build && npm test	36 passed, 5 skipped, consistent with design
live_golden-dataset_test.txt	npm run test:golden	4/4 live tests passed, 227.93s total
automatedpr.txt	npm run dev -- airaamane simple-todo-app 2 (POST_PR_COMMENT=true)	PR comment confirmed posted live
sanitychecks.txt	ls reports/; jq .summary; grep mermaid	All 9 required files present; mermaid block confirmed

Table 9. Core Deliverables Verification (Part 1 and Part 2)

#	Deliverable	Implementing File(s)	Verification
1	MCP server configuration for GitHub and ESLint	src/config/mcp.config.ts	Code inspection; tsc clean; confirmed live (4 real GitHub MCP sessions)
2	Three specialized subagents with targeted prompts	src/agents/*.ts, src/prompts/*.prompt.ts	Code inspection; orchestrator.test.ts; live findings in PR #1
3	Main orchestrator coordinating subagents	src/orchestrator.ts	9 mocked tests, all passing; confirmed live across 4 PR reviews
4	Aggregate results into validated ReviewReport	src/orchestrator.ts, src/types/report-types.ts	28 schema tests, all passing; live reports validated
5	Error handling: retry with backoff and timeout	src/utils/error-handler.ts	Code inspection; live retry-after-timeout observed on PR #1
6	Rate limiting (requests, tokens, concurrency)	src/utils/rate-limiter.ts	Code inspection; constructor test
7	CLI with argument and environment validation	src/main.ts	Code inspection; confirmed live across 4 real invocations
8	Comprehensive automated test suite	tests/*.test.ts	41 tests total: 36 passing by default, 4 additional passing live
9	Multi-format reports (JSON, Markdown, HTML)	src/main.ts + src/utils/report-generator.ts	Confirmed live: all 9 required files present with correct names

Table 10. Codebase Composition by Module

Module	Lines of TypeScript	% of Codebase
Tests (tests/)	671	27.3%
Utilities (src/utils/)	565	23.0%
Orchestrator (src/orchestrator.ts)	330	13.4%
Prompts (src/prompts/)	232	9.4%
CLI (src/main.ts)	186	7.6%
Types (src/types/)	179	7.3%
Agents (src/agents/)	65	2.6%
MCP Config (src/config/)	49	2.0%
Total	2,461	100%

Table 11. Rubric Cross-Reference

Rubric Category	Requirement Summary	Evidence
System Configuration	Env vars validated on startup; no hardcoded secrets	src/main.ts lines validating ANTHROPIC_API_KEY, ANTHROPIC_MODEL, GITHUB_TOKEN
MCP Servers	GitHub and ESLint configured with correct transport/command/args/env	src/config/mcp.config.ts
Subagent Definitions	3 agents with description, model: inherit, Skill tool	src/agents/*.ts
Prompt Design	Prompts reference schema and focus areas explicitly	src/prompts/*.prompt.ts
Skills Integration	At least one subagent invokes Claude Skills	code-quality-analyzer invokes 4 skills
Orchestrator Logic	query() used; Task tool + all 3 agents registered	src/orchestrator.ts Options object
Aggregation & Validation	outputFormat + Zod validation; graceful failure	ReviewReportSchema.safeParse() in orchestrator.ts
Multi-format Reports	report.json, .md, .html generated	src/main.ts + ReportGenerator
Testing	Unit tests cover schemas and utilities; pass cleanly	41 tests defined; 36 passing by default, 4 additional passing live
Error Handling	withRetry (backoff) and withTimeout implemented	src/utils/error-handler.ts
Rate Limiting	acquire, canProceed, pruneOldRecords implemented	src/utils/rate-limiter.ts
CLI Entry Point	Argument validation, orchestrator invocation, clean errors	src/main.ts
Code Quality	Strict TypeScript, no excessive any, organized structure	tsc --noEmit clean; modular src/ layout

9. Value Statement

The value of this project is best understood at three levels: as a working tool that could plausibly be adopted by a real engineering team, as a demonstration of production-grade agentic AI engineering practice, and as a template that generalizes to problems well beyond code review specifically. Each of these is worth stating explicitly and separately, because a project can be valuable along one axis without being valuable along the others, and the strongest case for this submission is that it holds up along all three simultaneously.

As a working tool, the system directly addresses the three failure modes described in the Problem Statement: inconsistent security review, silent test-coverage erosion, and deferred technical debt, by applying three consistently available, non-fatigable analytical lenses to every file in every pull request it is run against, at a cost and latency (a single orchestrator call plus up to three subagent calls per file, rate-limited and retried automatically) that is compatible with being run as a routine, even automatic, step in a real development workflow rather than an occasional manual audit. Its automated PR-commenting capability specifically closes the loop between analysis and action: a finding that only exists in a local `reports/` folder helps no one, but a finding posted directly onto the pull request under review is visible to exactly the people making the decision to merge it, at exactly the moment that decision is being made.

As a demonstration of engineering practice, the project's value lies less in any single clever technique and more in the discipline of composing several well-understood, independently-testable techniques correctly: schema-first structured output validated at two separate layers, retry logic composed correctly with timeout logic rather than merely present in isolation, a rate limiter that proactively prevents throttling rather than reactively handling it, and a test suite that is honest about which of its claims are verified deterministically versus which require a live, monitored regression check. None of these techniques is individually novel, but their correct composition, and the discipline of writing tests that actually exercise the composition rather than only its individual pieces, is exactly the kind of engineering rigor that separates a production-viable AI system from a compelling demo that breaks the first time it encounters a transient network error or a malformed model response.

As a generalizable template, the architecture pattern demonstrated here, a single orchestrating agent with MCP-based access to external state, a Task-tool-based fan-out to narrowly-scoped subagents each carrying its own domain-specific skill set, and schema-validated aggregation of their results into a single structured artifact, is not specific to code review at all. The same pattern applies directly to any workflow that requires synthesizing multiple, genuinely distinct analytical perspectives on the same input under a machine-checkable output contract: a legal-document review system with subagents for compliance, liability, and plain-language clarity; a customer-support triage system with subagents for sentiment, technical classification, and priority scoring;

or a financial-document analysis system with subagents for risk, compliance, and summarization. The specific prompts, schemas, and MCP servers in this project are code-review-specific, but the orchestration skeleton around them is not, which is precisely what makes the underlying engineering pattern the more durable and more broadly valuable artifact than any single feature of the code-review use case itself.

Finally, there is direct educational and portfolio value in the fact that every claim made in this report is independently checkable against the actual source code: a reader can run `npx tsc --noEmit`, run `npm test`, open `src/orchestrator.ts` and confirm the `Task` tool is present in `allowedTools`, or open the golden-dataset manifest and confirm the three fixture files exist on disk. That checkability is itself a form of value distinct from the system's functional behavior, because it is what allows a technical evaluator to trust the report's conclusions without having to take them on faith.

10. Limitations

An honest account of this project's limitations remains as important to a competitive submission after live validation as it was before, and this section has been revised to reflect what the live evidence shows rather than repeating limitations that live evidence has since resolved. The system has now been run end to end against three real pull requests, the golden-dataset regression suite has been executed live with all four tests passing, and the Automated PR Commenting standout has posted a real comment to a real pull request; the limitations below are the ones that remain after that evidence, not before it.

The most significant observation from the live run is that two of the three required pull requests, PR #2 and PR #3, returned zero analyzed files and a resulting score of 0. The system handled this gracefully, producing valid, schema-conformant reports rather than crashing, but it means the depth of evidence about the Code Quality Analyzer, Test Coverage Analyzer, and Refactoring Suggester's actual analytical output currently rests most heavily on a single pull request, PR #1, which did produce a rich, multi-file result. A useful next step would be confirming, directly on GitHub, exactly what PR #2 and PR #3 changed, and if their diffs genuinely contain no reviewable source files, selecting two additional PRs with substantive code changes to demonstrate the full analytical pipeline three times over rather than once.

Second, PR #1's total measured duration of 771.4 seconds exceeded the orchestrator's default single-attempt timeout of 600 seconds while still completing successfully, which is consistent with the retry-after-timeout path engaging in production, a positive sign for resilience, but also a concrete signal that the default 10-minute timeout and 2-retry budget are close to their practical ceiling for a substantive, multi-file review. A production deployment reviewing larger or more complex pull requests than this project's test cases would likely need to raise `timeoutMs` or reduce the default `maxFiles` per review to stay comfortably within a single attempt.

Third, the system's reliability is fundamentally bounded by the reliability of the underlying language model's instruction-following, and no amount of prompt engineering or schema constraint eliminates this dependency entirely, it only narrows the space in which failures can occur. A schema-invalid response is caught and surfaces as a clear `ReviewError`, but a schema-valid response that is nonetheless substantively wrong, a missed vulnerability, an incorrectly assessed severity, or a refactoring suggestion that does not actually compile, is not something the current validation layer can detect, because Zod validates shape and type, not semantic correctness. The golden-dataset suite mitigates this for the Code Quality Analyzer specifically, and its live pass is meaningful evidence, but it is not, and cannot be, a comprehensive guarantee of analytical quality across the effectively unbounded space of real-world code.

Fourth, the golden-dataset regression suite, though now proven live, covers only the Code Quality Analyzer; the Test Coverage Analyzer and Refactoring Suggester have no equivalent fixture-based regression check. Extending an equivalent golden dataset to a known test-coverage gap and a known refactoring opportunity would close this gap and bring the same live-verified confidence already achieved for security-vulnerability detection to the other two subagents.

Finally, the GitHub MCP server integration relies on the community-maintained `@modelcontextprotocol/server-github` reference package, which as of this writing carries an upstream deprecation notice in favor of GitHub's own officially maintained `github/github-mcp-server` project. The community package remains functional and was directly exercised, successfully, in every live run documented in this report, but a future maintenance pass migrating to the officially maintained server, which uses a different tool-naming convention, would be a reasonable follow-up rather than a purely optional improvement.

11. Conclusion

This project set out to demonstrate that a well-architected, multi-agent system built on Anthropic's Claude Agent SDK can consistently apply the several distinct analytical lenses that real code review requires, security and quality, test coverage, and structural refactoring opportunity, without any of those lenses degrading under the fatigue, inconsistency, or time pressure that limits purely human review at scale. The completed implementation supports that thesis concretely, and it is no longer a claim resting on architecture and mocked tests alone: a full live run, executed with real Anthropic API and GitHub credentials, has since confirmed the system working end to end against three real pull requests, with all nine required report files produced under their exact required filenames.

Every one of the nine core deliverables specified across the project's two phases was implemented and is independently verifiable, and the live run adds a further layer of confirmation on top of the original code-level and mocked-test verification: the MCP configuration for both required servers functioned correctly against the real GitHub API across four live reviews, three distinct subagents were explicitly invoked and returned schema-valid results on the one pull request with substantive file changes, the orchestrator correctly aggregated and validated its output every time, and the automated test suite's previously-skipped golden-dataset tests have now been executed live with all four passing.

All four suggested standout features were not only implemented but directly exercised live: Automated PR Commenting posted a real, confirmed comment to a real pull request; the Interactive CLI completed every run to a clean success state; the Regression Testing Suite passed 4 of 4 live tests against the real, configured model; and the Agent Execution Traces standout was confirmed present, via direct inspection, in the actual generated Markdown report. Each of these is documented with a reproducible terminal transcript in Section 8 rather than asserted from design intent alone.

This report has also been candid about what the live evidence itself reveals as remaining open questions rather than treating a successful run as proof of a finished, closed system: two of the three required pull requests returned zero analyzed files, which is most plausibly an artifact of those PRs' specific diff contents rather than a system defect, and the one substantive review that did run took long enough to demonstrate the retry-after-timeout path engaging in practice, a useful signal for tuning timeout and file-count defaults in a heavier production deployment. These are presented as the specific, concrete next steps suggested by real execution data, which is a stronger foundation for future work than limitations inferred from an untested design.

In sum, the project delivers a complete, tested, and thoughtfully extended implementation of the assigned Enterprise Multi-Agent Code Review Orchestrator: every required file is

implemented, every rubric criterion identified in the project's own instructions document is met and cross-referenced to the code that satisfies it, every standout suggestion is realized as working functionality, and, as of this update, every one of those claims has been checked against a real, live, credentialed execution of the system rather than left as an untested assertion.

12. References

All references below were independently verified as live, accessible sources during the preparation of this report.

[1] Anthropic. (2026). Claude Agent SDK overview. Claude Developer Platform Documentation. https://platform.claude.com/docs/en/agent-sdk/overview

[2] Model Context Protocol. (n.d.). Model Context Protocol: Specification and documentation. https://modelcontextprotocol.io/

[3] Model Context Protocol. (n.d.). @modelcontextprotocol/server-github [npm package]. https://www.npmjs.com/package/@modelcontextprotocol/server-github

[4] GitHub, Inc. (n.d.). github/github-mcp-server: GitHub's official MCP server. https://github.com/github/github-mcp-server

[5] OpenJS Foundation, ESLint. (n.d.). MCP server setup. ESLint documentation. https://eslint.org/docs/latest/use/mcp

[6] ESLint. (n.d.). @eslint/mcp [npm package]. https://www.npmjs.com/package/@eslint/mcp

[7] Hacks, C. (n.d.). Zod: TypeScript-first schema validation with static type inference. https://zod.dev/

[8] Terdell, S. (n.d.). zod-to-json-schema [npm package]. Converts Zod schemas to JSON Schemas. https://www.npmjs.com/package/zod-to-json-schema

[9] OWASP Foundation. (2021). OWASP Top Ten 2021: The standard awareness document for web application security. https://owasp.org/Top10/2021/

[10] Mermaid.js. (n.d.). Mermaid: Generation of diagrams and charts from text in a markdown-like syntax. https://mermaid.js.org/

[11] Vitest. (n.d.). Vitest: A next generation testing framework powered by Vite. https://vitest.dev/

[12] Sindresorhus. (n.d.). ora [npm package]: Elegant terminal spinner. https://www.npmjs.com/package/ora

13. Addendum: Additional Live Pull Request Validation

Section 10 (Limitations) noted that two of the three originally required pull requests, `airaamane/simple-todo-app` PR #2 and PR #3, returned zero analyzable files during the live run documented in Section 8, leaving the bulk of this report's direct evidence for the subagents' substantive analytical output resting on a single pull request. To close that specific gap with additional, independent evidence, two further live pull requests were subsequently authored and reviewed end to end against a second, unrelated public repository, `eaamankwah/covid19-data-analysis-and-visualization-project`, using the exact same unmodified CLI, orchestrator, and subagent configuration described throughout this report.

Unlike the original three pull requests, both of these two new pull requests were deliberately authored to contain concrete, verifiable code review targets, in the same spirit as the golden-dataset fixtures described in Section 6, so that a successful run would be judged not merely by whether the system completed without crashing, but by whether its findings were substantively correct. PR #1 added two Python helper functions to `covid19-data-analysis-and-visualization.py` containing a hardcoded API-style credential, an unvalidated file-path parameter, no error handling around file I/O, and roughly ninety percent code duplication between the two functions. PR #2 added a small inline JavaScript snippet to `covid19-data-analysis-and-data-visualization.html` that reads a `country` value directly from the page's URL query string and assigns it to an element's `innerHTML` without sanitization, a textbook DOM-based cross-site-scripting pattern.

Both pull requests were reviewed successfully via `npm run dev -- eaamankwah covid19-data-analysis-and-visualization-project <N>`, and each run produced all three required report files (`.json`, `.md`, `.html`) under their exact expected filenames, with no crashes, no schema-validation failures, and no execution errors of any kind. Table 12 reproduces the measured outcome of each run.

Table 12. Additional Live Pull Request Review Results (eaamankwah/covid19-data-analysis-and-visualization-project)

PR	Files Analyzed	Overall Score	Critical Issues	High-Priority Test Gaps	Refactoring Opportunities	Duration	Primary File Reviewed
#1	1	35 / 100	1	8	7	240.6 s	covid19-data-analysis-and-visualization.py
#2	1	25 / 100	1	4	6	268.7 s	covid19-data-analysis-and-data-visualization.html

Both runs surfaced substantive, correctly differentiated findings rather than generic or padded output. On PR #1, the Code Quality Analyzer correctly identified the planted hardcoded Plotly API key as a critical severity, security-category finding, explicitly citing OWASP A07:2021 (Identification and Authentication Failures), and separately flagged the unvalidated file path parameter, the missing file-I/O error handling, and the absence of column-existence validation, while the Refactoring Suggester independently identified the roughly ninety percent code duplication between the two near-identical functions and proposed a single consolidated function with optional parameters as the fix, together with a worked before-and-after code example. The Test Coverage Analyzer identified sixteen distinct untested paths across both functions, including the exact IndexError risk that would occur if `sum_cols` were passed as an empty list, a defect that was present in the planted code but not called out in the original commit message.

On PR #2, the Code Quality Analyzer correctly identified the DOM-based cross-site-scripting vulnerability as a critical severity, security-category finding, explaining precisely how a crafted URL such as `?country=<img src=x onerror=alert(document.cookie)>` could execute arbitrary script in a visitor's browser, and recommended replacing the vulnerable `innerHTML` assignment with `textContent`. The same finding was independently corroborated by the Test Coverage Analyzer, which proposed specific test cases covering both a raw `<script>` payload and a URL-encoded equivalent, and by the Refactoring Suggester, which supplied a complete before-and-after rewrite replacing `var` declarations and string concatenation with `const`, template literals, and the safe `textContent` API. Both subagents' independent identification of the same root-cause vulnerability, arrived at through separate prompts and separate reasoning paths, is itself a meaningful reliability signal beyond what either finding would demonstrate alone.

Taken together with the original three-pull-request evidence in Section 8, this addendum brings the total number of live, credentialed pull request reviews documented in this report to five, spanning two independent public repositories and three distinct file types (Python, JavaScript, and JSON/CSV-adjacent data-analysis code), and it specifically supplies the substantive, multi-file, multi-finding evidence that PRs #2 and #3 of the original three could not, since those two returned no analyzable files. No change was made to the orchestrator, the subagents, the prompts, or the schemas to produce this additional evidence; the same unmodified system that produced Section 8's results was simply pointed at a different, independently authored set of pull requests and asked to review them under its existing configuration, which is itself the intended mode of operation for a general-purpose code review tool rather than a special accommodation made for this report.